软件工程管理过程总结报告

项目名称：“智学引擎”智能学习系统
小组名称/编号： G09
填写日期： 2025 年 12 月 27 日

1.过程组织与人力资源

1.1 角色与职责矩阵

角色	成员姓名	主要职责描述
项目经理、首席架构师、算法工程师	陈子涵	1. 设计 Router/Service/DAO 分层架构与 API 规范。 2.攻坚 RAPTOR 递归树状索引算法实现，实现 reranker 等核心算法。 3. 主持 Sprint 评审，管理 Backlog 与代码封板节点。 4. 负责核心代码 Code Review 与技术风险评估。
后端工程师、测试工程师	邓佳骏	1. 后端负责资料管理中心与外部文献检索模块的构建与优化。解决 Python GIL 锁问题，设计异步线程池优化 CPU 密集型任务等等。 2. 负责系统测试，负责系统压测与服务器资源监控等等。
后端工程师、设计总监	方李庭	1. 后端负责资料管理中心与外部文献检索模块的构建与优化。编写 PDF/Markdown 清洗脚本，处理复杂排版与乱码等等。 2. 负责总体界面的设计，

		前端代码的开发，协助进行初期竞品调研与功能定义。
后端工程师、交互工程师	**郭云海**	1. 后端负责智能问答中心的构建与优化，优化大模型提示词，解决 JSON 输出幻觉问题等等。 2. 负责优化与提升用户体验，攻克 SSE 协议，实现前后端流式响应等等。
后端工程师、算法工程师、前端工程师	**魏文俐**	1. 后端负责智能问答中心的构建与优化，设计 BM25 + Vector 的双路召回策略等等。 2. 负责探索前沿分块算法实现，重构索引模块，引入语义分块替代固定切分。 3. 进行前端界面的开发与优化。

1.2 团队能力建设

在项目启动和中期，团队进行了以下针对性培训：

1. RAPTOR 算法研读会（中期）：由组长主讲 ICLR 2024 论文《RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval》，统一全员对递归树状索引的理解。

2. Next.js 学习（初期）：鉴于全员都更擅长后端背景，团队进行了为期 2 天的 Next.js + Tailwind CSS 学习，利用 Shadcn UI 组件库弥补前端短板。

3. SSE 协议与异步编程（中期）：针对流式响应开发难点，组织学习了 HTTP/2 Server-Sent Events 协议及 Python asyncio 标准库。

2. 软件过程实施情况

2.1 需求管理

- **目标客群与聚焦：**

本项目锁定高校在读学生及科研人员。 特定场景聚焦于：

1.专业课深度复习：需处理数百页非结构化 PDF 课件，且对“进程控制块”、“死锁”等专业术语的召回精度要求极高 。

2.学术长文研读：需突破 LLM 上下文限制，理解整篇论文的宏观逻辑架构 。

3.连贯式知识溯源：支持多轮追问与精确引用的知识溯源，拒绝模型“幻觉” 。

- **需求变更控制：**

共发生 2 次大的需求变更，均经过 Sprint Review 评审通过。变更流程执行情况如下：

第一次变更（中期优化 - 11 月中旬）：

背景：MVP 测试显示，纯向量检索对专有名词召回率低，且系统无法记住上一轮对话 。

变更内容：

1.检索策略升级：从“单一向量检索”变更为 “混合检索 (Hybrid Search) + 重排序 (Rerank)”，结合 BM25 解决术语匹配问题 。

2.上下文策略升级：引入 Sentence Window (句子窗口) 检索，减少断章取义 。

3.记忆模块重构：引入 SQLite 替代内存存储，实现多轮对话持久化 。

执行：此变更确保了中期答辩时系统具备了“高可用性”。

第二次变更（终期算法攻坚 - 12 月上旬）：

背景：中期答辩反馈研究前沿算法，且用户反馈生成摘要等待时间过长 。

变更内容：

1.核心算法深化：引入 RAPTOR 递归树状索引与语义分块 (Semantic Chunking)，提升宏观理解力 。

2.交互与性能重构：全栈改造为 SSE 流式响应，并增加 SQLite 缓存与异步并发机制 。

执行：此变更确保了结题时的技术先进性与用户体验。

2.2 项目计划与监控

- **生命周期模型选择：** (勾选并简述理由)
  - 瀑布模型
  - 迭代/螺旋模型
  - √ 敏捷/Scrum
  - *理由：*项目涉及大量前沿 RAG 技术的探索（如从简单的 Chunking 到 Semantic Chunking，从 Vector 到 Hybrid），需求与技术方案具有高度不确定性。敏捷

模式允许我们以 2 周为周期快速试错，根据 MVP 和中期答辩的反馈灵活调整算法策略，避免了瀑布模型“一条道走到黑”的风险 。

- **里程碑设置：**

我们设定了严密的里程碑，清晰划分为“基础构建”、“深度优化”与“算法攻坚”三个阶段：

1.项目启动（10.13）：完成选题论证，确定 FastAPI + Next.js + LlamaIndex 技术栈 。

2.MVP 原型交付（11.07）：跑通“上传-解析-检索”最小闭环，验证技术可行性 。

3.V2.0 中期深度优化（11.20 ）：

完成混合检索与重排序上线，解决“查不准”问题。

完成 TreeSummarize 递归摘要，突破长文档限制 。

通过中期答辩，确立了系统的核心竞争力。

4.V2.1 终期算法攻坚（12.08）：

语义分块与流式响应上线，解决“切分生硬”与“响应慢”问题 。

5.最终交付与封板（12.20）：完成并发缓存优化，代码冻结，输出测试报告 。

2.3 风险管理

风险类别	具体风险描述	采取的应对措施/预案	状态 (已规避/已发生/进行中)
技术质量风险	**“答非所问”风险：**MVP 版本中，用户搜索“进程控制块”等术语时，向量检索匹配度极低，导致回答幻觉严重 。	**混合检索架构：**实施 Hybrid Search，并行执行 BM25（关键词）与 Vector（语义）检索，并引入 BGE-Reranker 进行二次精排，确保 Top-5 结果精准	已解决
技术可行性风险	**长文档处理失效：**LLM 上下文窗口限制导致无法一次性读取 50 页论文，	**递归摘要策略：**落地 TreeSummarize 算法，采用 Map-Reduce 思想分块	已解决

	系统只能“读头不读尾” 。	摘要再递归汇总，成功突破 Token 限制 。	
用户体验风险	**“响应假死”风险**：长文档摘要生成耗时超过 20 秒，前端无反馈，用户以为系统崩溃 。	**流式与缓存双管齐下：** 1. 改造 **SSE** 协议实现打字机效果 。 2. 引入 **SQLite 缓存**，二次访问直接返回结果（0 秒耗时） 。	已解决
协作风险	**代码冲突阻塞**：全员参与后端开发，5 人同时修改 python 导致合并冲突频发，进度受阻 。	**架构解耦管理**：将单体架构重构为 Router/Service/DAO 分层架构，并确立 API **契约先行** 原则，物理隔离开发边界 。	已规避

2.4 配置管理与过程资产

- **版本控制策略：**

使用 Git 进行全流程管理，采用 Git Flow 策略：

1. main：生产分支，仅在中期答辩和结题答辩前合并稳定版本。

2. dev：日常开发集成分支。

3. feat/hybrid-search：中期优化的混合检索功能分支 。

4. feat/semantic-chunking：终期优化的语义分块功能分支 。

5. ...

- **过程资产库建设：**

已沉淀以下资产供复用：

1. 经验教训库：记录了“Python 异步阻塞 Event Loop”的排查与解决方法等 。

2. 可重用组件库：

HybridRetriever：封装好的混合检索器类 。

SSEGenerator：通用的流式响应生成器 。

开发工具库：init_db.sql（SQLite 初始化脚本）与 Docker Compose 配置。

标准过程库：制定了 API 接口定义模版（Pydantic Schema）与 Code Review 清单。

2.5 过程与产品质量保证 (PPQA & Verification)

- **静态测试：**

Code Review：针对核心算法（如 HybridRetriever 的权重分配、SemanticSplitter 的阈值设定）进行了两轮严格的代码审查 。

- **动态测试：**

1. A/B 测试：

对比了“纯向量检索”与“混合检索+重排序”的效果。结果显示，引入优化后，专有名词召回率从 60% 提升至 95% 。

2. 性能压测：

针对 并发加速 优化，模拟 50 并发请求。测试表明，开启 use_async=True 后，长文档处理效率提升了 3 倍 。

3. 场景测试：

验证 Sentence Window 策略：检查检索结果是否包含了完整的上下文句子，而非被截断的片段 。

3. 度量与分析

度量指标	统计值
项目总工时	600 人/时
代码行数 (LOC)	8, 500 行
发现缺陷数	42 个
文档产出量	9 份
需求完成率	100 %

4. CMMI 成熟度自评

参考标准：

- **初始级：** 过程无序，进度和质量依赖个人发挥。
- **受管理级：** 有计划、有跟踪，能复用同类项目的管理经验。
- **已定义级：** 拥有组织级的标准流程和文档库，过程标准化。

自评等级： 受管理级

自评理由：

1. 有计划与跟踪：项目全程依据 Scrum 流程进行，有明确的 Sprint 计划、隔周站会跟踪以及里程碑评审，并非无序开发。

2. 需求与配置管理：建立了完整的需求变更流程和严格的 Git 版本控制策略，确保了工作产品的完整性。

3. 质量保证：实施了 Code Review 和系统测试，并在过程中识别和管理了风险。

4. 未达到“已定义级”原因：虽然团队内部有标准，但尚未形成组织级（跨团队）的标准化过程资产库，流程剪裁的依据主要基于团队习惯而非组织数据。

5. 持续优化与改进

5.1 根本原因分析(RCA)

- **成功经验分析：**

现象：尽管全员参与后端，但前后端联调效率极高，未出现重大接口扯皮。

根本原因：实施了 API First（接口契约先行） 策略。在写代码前，先在 Apifox 定义好入参出参结构（Pydantic Model）。

推广价值：对于跨职能团队或技能栈偏科的团队，严格的接口契约是降低协作成本的关键。

- **问题与不足分析：**

现象：Sprint 3 初期，RAPTOR 算法集成导致后端服务频繁假死，阻塞了前端开发。

根本原因：对 Python asyncio 的 GIL（全局解释器锁） 机制理解不深，错误地在异步函数中执行了同步的 CPU 密集型计算（K-Means），导致事件循环被阻塞。

5.2 技术与过程创新

- **过程管理创新：**

全员全栈化协作：在面对大家都更擅长后端的情况时，没有机械地等待，而是通过 全体成员一起学习的方式，实现了全员全栈开发，极大提升了人力资源的流动性。

- **技术价值深度：**

1. 超越 CRUD：本项目不是简单的增删改查系统。核心价值在于实现了 RAPTOR 递归树状索引，涉及高维向量聚类、图结构构建与递归摘要生成。

2. 工业级优化：引入了流式响应（SSE）与多级缓存架构，解决了 AI 应用常见的响应慢、成本高问题，具有较高的工程参考价值。